

```

        y = 0
    elif y > self.maxY:
        y = self.maxY

    self.image.setLoc((x, y))
    return self.image.getRect()

❸ def draw(self):
    self.image.draw()

```

Listing 16-10: The `Player` class

The `__init__()` method ❶ loads the `Player` icon image and sets up a number of instance variables for use later.

The `update()` method ❷ is called in every frame by the `Play` scene. The basic idea is to show the `Player` icon at the mouse location, which is passed in. We do a few checks to ensure that the icon remains within the rectangle of the playable area. In every frame, the `update()` method returns the updated rectangle of the `Player` icon so the main `Play` code in [Listing 16-6](#) can check if the `Player`'s rectangle intersects with any `Baddie` or `Goodie`.

Finally, the `draw()` method ❸ draws the `Player` icon at the new location.

The use of the `Goodie` manager, the `Baddie` manager, and the `Player` object clearly demonstrates the power of OOP. We can just send messages to these objects, asking them to update or reset themselves, and they do whatever they need to do in response. The `Goodie` and `Baddie` managers pass these messages on to all the `Goodies` and `Baddies` that they manage.

File: `Dodger/SceneHighScores.py`

The `High Scores` scene displays the top 10 high scores (and the names of the players) in a table. It also allows a user who has scored in the top 10 to optionally enter their name and score into the table. The scene instantiates a `HighScoresData` object to manage the actual data, which includes reading and writing the data file. This allows the `High Scores` scene to update the table and to respond to requests from the `Play` scene for the current high and low scores in the table.